

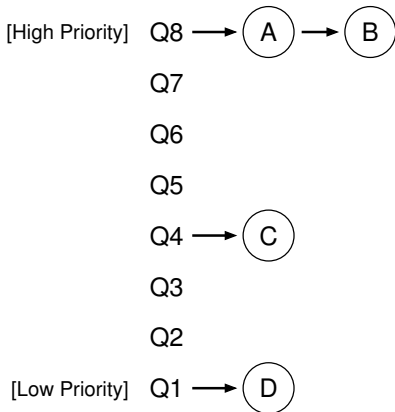
CSL 301

OPERATING SYSTEMS

Lecture 5

The Multi-Level Feedback Queue

Instructor
Dr. Dhiman Saha





Recall

- ▶ **Interactive** jobs care about **response** time
- ▶ **Batch** jobs care about **turnaround** time

Scheduling **without** perfect knowledge

How can we design a scheduler that both minimizes response time for interactive jobs while also minimizing turnaround time **without** a priori knowledge of job length?

Recall

Algorithms like Round Robin **reduce response time** but are terrible for turnaround time. Why?

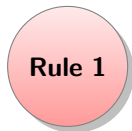
- ▶ How can the scheduler **learn, as the system runs, the characteristics of the jobs it is running**, and thus make better scheduling decisions?

Idea

Multiple levels of round-robin

- ▶ Due to Corbato et al. [1962]
- ▶ Later awarded Turing Award
- ▶ MLFQ has a number of distinct queues
- ▶ Each assigned a different **priority level**
- ▶ MLFQ uses **priorities** to decide which job should run at a given time

A job with higher priority is chosen to run.



Rule 1

If $Priority(A) > Priority(B)$,
A runs (B doesn't).

Rule 1

If $Priority(A) > Priority(B)$,
A runs (B doesn't).

Rule 2

Rule 1

If $Priority(A) > Priority(B)$,
A runs (B doesn't).

Rule 2

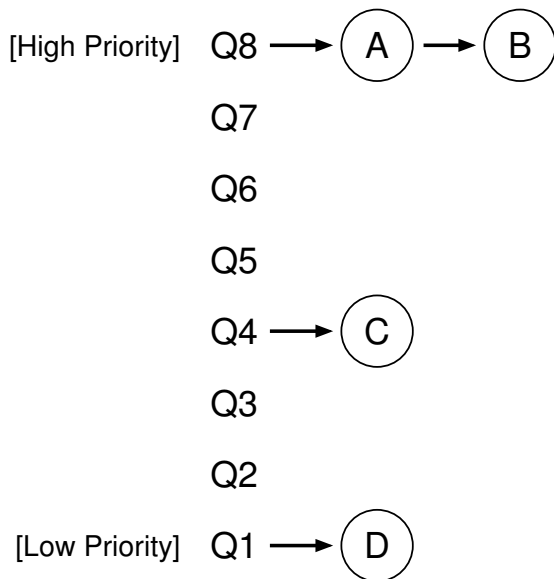
If $Priority(A) = Priority(B)$,
A & B run in RR

Rule 1

If $Priority(A) > Priority(B)$,
A runs (B doesn't).

Rule 2

If $Priority(A) = Priority(B)$,
A & B run in RR



The key to MLFQ scheduling therefore lies in how the scheduler sets priorities

- ▶ Approach 1: User supplied. e.g. `nice`
- ▶ Approach 2: Observed behavior (learn from history)

Rather than giving a fixed priority to each job, MLFQ **varies** the priority of a job based on its observed behavior



Rule 3

When a job enters the system,
it is placed at the highest
priority (the topmost queue).

Rule 3

When a job enters the system,
it is placed at the highest
priority (the topmost queue).

Rule 4a

Rule 3

When a job enters the system,
it is placed at the highest
priority (the topmost queue).

Rule 4a

If a job uses up an entire
time slice while running,
its priority is reduced (i.e.,
it moves down one queue)

Rule 3

When a job enters the system, it is placed at the highest priority (the topmost queue).

Rule 4a

If a job uses up an entire time slice while running, its priority is reduced (i.e., it moves down one queue)

Rule 4b

Rule 3

When a job enters the system, it is placed at the highest priority (the topmost queue).

Rule 4a

If a job uses up an entire time slice while running, its priority is reduced (i.e., it moves down one queue)

Rule 4b

If a job gives up the CPU before the time slice is up, it stays at the same priority level

Rule 3

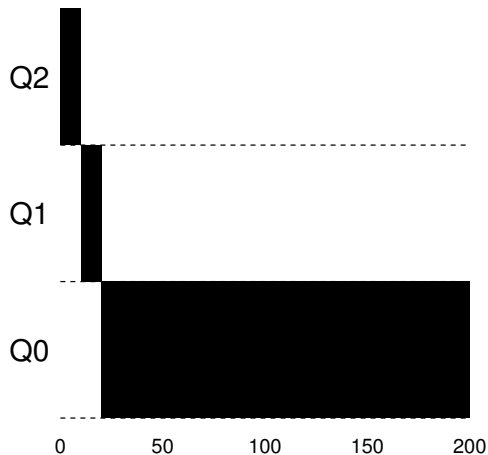
When a job enters the system, it is placed at the highest priority (the topmost queue).

Rule 4a

If a job uses up an entire time slice while running, its priority is reduced (i.e., it moves down one queue)

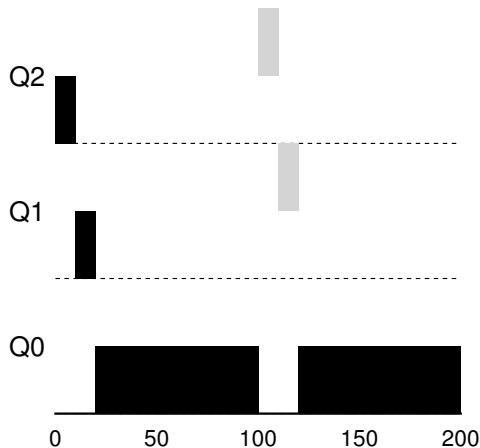
Rule 4b

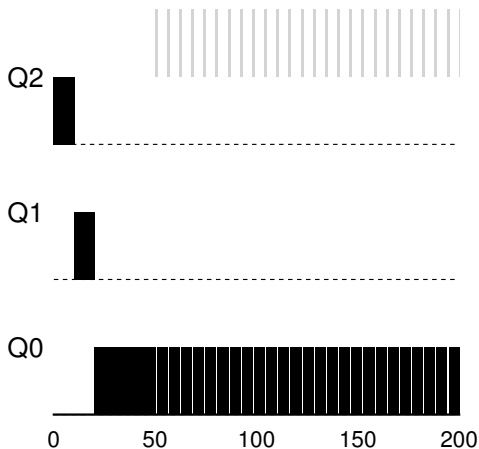
If a job gives up the CPU before the time slice is up, it stays at the same priority level



Example 2

Along Came An Interactive Job





A Mixed I/O-intensive and CPU-intensive Workload

Starvation

What if there are **too many** interactive jobs in the system?

Gaming the scheduler

What if an user tricks the scheduler into giving you more than your fair share of the CPU?

Change in behavior of a job

Is the current approach adaptive?

Rule 5

After some time period S ,
move all the jobs in the sys-
tem to the topmost queue

Solved

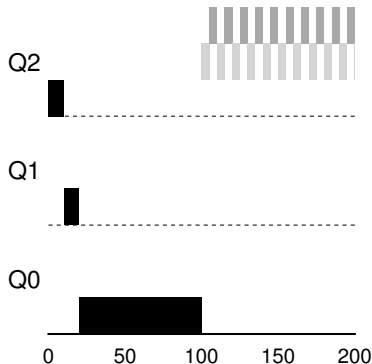
- ▶ Starvation
- ▶ Adapting to change in behavior

Why?

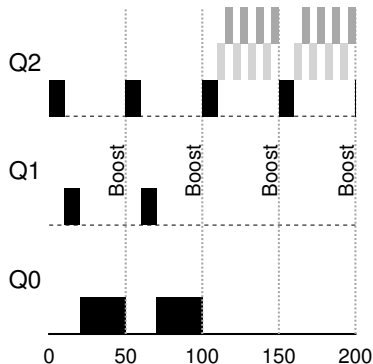
Still Unsolved

- ▶ Gaming the scheduler.

► Value of $S = 50ms$



Without Boost



With Boost

How to decide the value of S ?

The Voo-Doo Constant (coined by John Ousterhout)

- ▶ Too high, and long-running jobs could starve
- ▶ Too low, and interactive jobs may not get a proper share of the CPU.

Rule 4a

If a job uses up an entire time slice while running, its priority is reduced (i.e., it moves down one queue)

Rule 4b

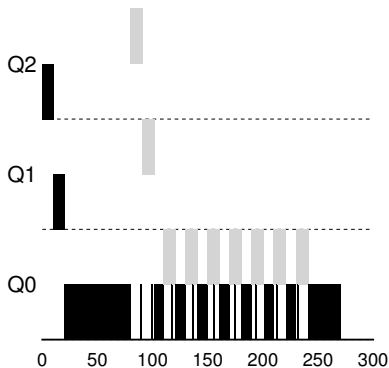
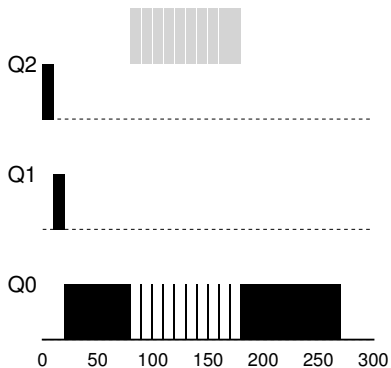
If a job gives up the CPU before the time slice is up, it stays at the same priority level

Rule 4

Once a job uses up its time allotment at a given level, its priority is reduced

- ▶ Perform better accounting of CPU time at each level of the MLFQ
- ▶ Keep track of actual time spent on the CPU
- ▶ Demote once allotted time is over
- ▶ Regardless of how many times the job has given up the CPU

Gaming Tolerance (GT)



- ▶ How many queues should there be?
- ▶ How big should the time slice be per queue?
- ▶ How often should priority be boosted in order to avoid starvation and account for changes in behavior?

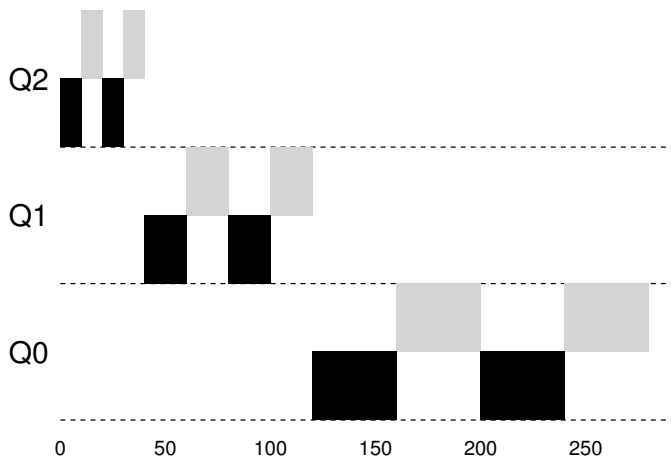
No easy answers to these questions

Varying time-slice length across different queues

- ▶ The high-priority queues are usually given short time slices; they are comprised of interactive jobs,
- ▶ The low-priority queues, in contrast, contain long-running jobs that are CPU-bound; hence, longer time slices work well

Other features

advice on setting priorities e.g. `nice` command on UNIX



Rule 1

If $Priority(A) > Priority(B)$,
A runs (B doesn't).

Rule 2

If $Priority(A) = Priority(B)$,
A & B run in RR

Rule 3

When a job enters the system, it
is placed at the highest priority

Rule 4

Once a job uses up its
time allotment at a given
level, its priority is reduced

Rule 5

After some time period S ,
move all the jobs in the sys-
tem to the topmost queue

Idea

Lottery Scheduling

Approach:

- ▶ Processes are allotted lottery tickets
- ▶ Whoever wins runs
- ▶ Higher priority implies more tickets